

Linux Operating System Assignment-1

1.What If There Were No Operating System? Imagine a computer without an operating system. Identify the problems a user would face and investigate how OS features solve them. Demonstrate at least three features using Linux and present a “Without OS vs With OS” comparison.

Turning on a computer without an OS does not cause an explosion or total silence. The hardware wakes up and does its basic self-check (via BIOS/UEFI firmware), but then hits a dead end because there is no software to load, organize memory, manage files, or accept your commands. It is a fully awake machine with absolutely nothing meaningful to do.

What would actually happen?

1.No way to run multiple programs: Without an OS, the CPU can only do one thing at a time, given directly by the programmer. For example, we couldn't listen to music while typing a document. Every single program would need to control the hardware completely by itself, and only one could run at a time.

2.No file organization: There would be no concept of "files" or "folders". Data would just be raw bits sitting on the disk with no name, no structure, no way to find anything. We would have to remember exact physical disk locations to get your data back.

3.No memory management: Every program would have to directly manage physical memory (RAM) on its own. Two programs could easily overwrite each other's data by accident, causing crashes constantly.

4.No device communication (drivers): The keyboard, mouse, printer, Wi-Fi card .All speak different technical "languages." Without an OS, every single app would need built-in code to talk to every possible device directly. That is really inefficient.

5.No security or user accounts: Anyone could access any data. No permissions, no passwords, no protection between users or programs.

6.No easy way to install or run software: There would be no standard way to load a program into memory and run. It would require deep hardware knowledge just to start anything.

Following OS feature can solve this by:

Fix 1: Running Multiple Programs at Once: The OS quickly switches between programs, giving each one a tiny slice of CPU time. So it feels like they're all running together.

ps aux | head -5

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.2	0.6	20592	13452	?	Ss	00:51	0:40	/usr/lib/systemd/systemd --system --deserialize=10 splash
root	2	0.0	0.0	0	0	?	S	00:51	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	00:51	0:00	[pool_workqueue_release]
root	4	0.0	0.0	0	0	?	I<	00:51	0:00	[kworker/R-rcu_gp]

This shows a list of the top five active processes running on your Linux system. It combines `ps aux` to list all current processes with `head -5` to filter and display only the header line plus the first four process lines.

Fix 2: Organizing Your Files: The OS turns messy raw storage into something you can actually use — files and folders with names.

ls -l /home/file1

```
—(kali㉿kali)-[~/home] └─$ mkdir file1
```

```
└─(kali㉿kali)-[~/home] └─$ cd file1
```

```
└─(kali㉿kali)-[~/home/file1] └─$ touch file1.txt
```

```
└─(kali㉿kali)-[~/home/file1] └─$ ls
file1.txt
```

```
└─(kali㉿kali)-[~/home/file1] └─$
```

This works only because the OS keeps track of exactly where each file lives on the disk.

Fix 3: Managing Memory Properly: The OS decides how much memory each program gets, and makes sure they don't overlap.

free -h

total	used	free	shared	buff/cache	available
-------	------	------	--------	------------	-----------

Output shows things like total memory, used memory, and free memory — all tracked live by the OS. Without this, your programs would randomly grab memory space and crash into each other constantly.

Without OS vs With OS Comparison:

What	Without an OS	With an OS
Running programs	Runs only one program at a time, controlling everything itself	Many programs run together smoothly
Storage	Store raw data	Stored neatly organized files and folders
Memory	Programs fight over memory space, causing crashes	OS shares out memory safely for each program
Security	Anyone can access anything as no logins required and there is no protection	Passwords, permissions, and user accounts keep things safe
Ease of use	Need deep technical knowledge to do anything	Just click or type simple commands
Multitasking	Basically impossible	Built right in, works automatically

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

Lab role	Linux tools	Why
Penetration testing	Kali Linux	The classic — hundreds of attack tools ready to go, huge community support
Advanced/heavy pentesting	BlackArch	For when Kali's toolset isn't enough, and you're comfortable with Linux internals
Privacy-focused testing	Parrot OS	Same pentest muscle as Kali, but lighter and more privacy conscious
Anonymity/privacy work	Tails	Leaves zero trace, routes everything through Tor which is ideal for teaching privacy concepts
Security monitoring / blue team	Security Onion	Flip side of pentesting. This is for watching and defending, not attacking
Digital forensics / incident response	CAINE	Built specifically to investigate a system after something bad already happened

Kali Linux — The All-Rounder Hacker's Toolkit This is basically the most famous "hacking OS" out there. It comes pre-loaded with hundreds of tools for testing how secure a system is like password crackers, network scanners, and exploit frameworks. Made by Offensive Security, it is the go-to choice for penetration testing (legally trying to break into systems to find weaknesses before real hackers do).

Parrot OS — Kali's Lighter, Privacy-Friendly Cousin Very similar to Kali in what it offers (pentesting tools), but it is lighter on the computer's resources and also includes tools focused on privacy and anonymous browsing. Good if we want one OS that handles both hacking practice and staying private online.

BlackArch Linux — For the Tool Hoarders This one is huge in scale it has over 2,800 hacking/security tools bundled in. It is built on Arch Linux, which means it is more technical and needs some Linux experience to use comfortably. Best for advanced penetration testers who want maximum tool variety.

Tails (The Amnesiac Incognito Live System) — The Privacy Ghost Tails is built purely for privacy and anonymity. It routes all internet traffic through Tor (a network that hides your identity). It does not save anything on the computer once you shut it down. Boot it from a USB, use it, remove the USB, and it is like we were never there. Journalists, activists, and privacy-focused people use this a lot.

Security Onion — The Watchdog This tool is not for attacking systems . It is for defending and monitoring them. It is built for security monitoring and threat detection, packed with tools that watch network traffic and alert us when something suspicious happens. For example, CCTV cameras + alarm system for your network.

CAINE - (Computer Aided Investigative Environment) Specifically built for digital forensics. Investigating a hacked/compromised system after an attack already happened. It helps recover deleted files, analyze disk images, and build evidence, similar to how detectives investigate a crime scene, but digitally.

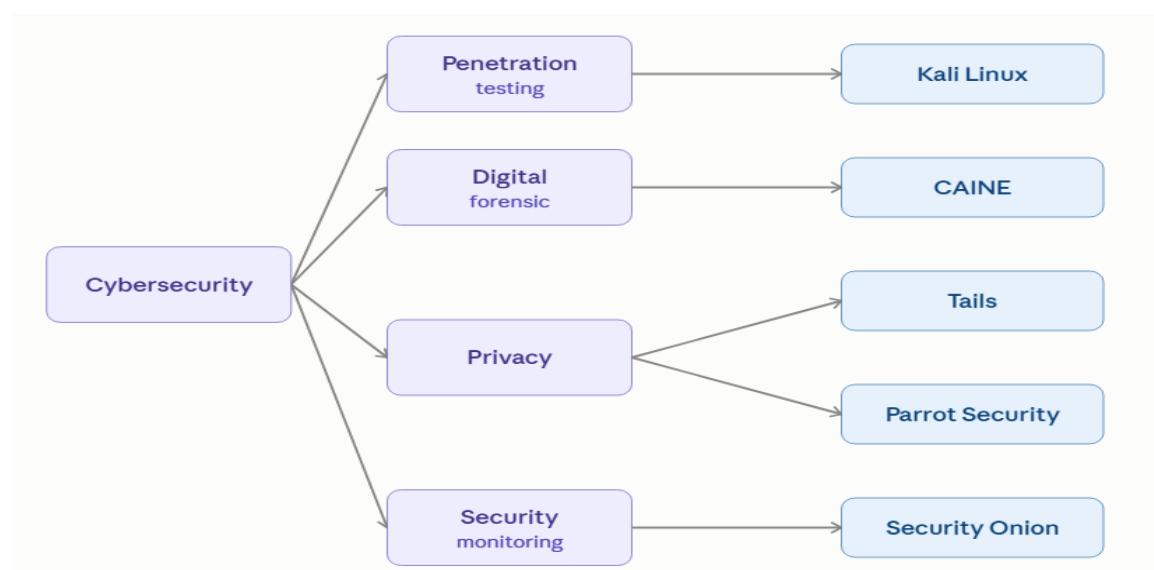
A real cybersecurity team doesn't just attack (red team), they also defend (blue team) and investigate after the fact. So a good lab setup usually mixes:

Offense: Kali or Parrot for students to practice attacking test systems

Defense: Security Onion to practice spotting those same attacks in real time

Forensics: CAINE for the "the attack already happened, now what?" scenario

Privacy: Tails to teach how attackers (or privacy-conscious users) stay hidden



3.Exit status: Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using \$? . Create a simple visual/table showing *Command* → *Result* *Exit Status* → *Meaning*. What can a script learn from an exit status?

Exit status is how a command silently tells the script if it worked or failed. And, a script follows that answer and reacts accordingly. Every time a command is run in Linux, once it finishes, it leaves behind a "report card" number, this is called the **exit status**. We can check it immediately after by typing **echo \$?**

Suppose, we get:

0 = "Everything went fine, succeeded"

Anything other than 0 (1, 2, 127, etc.) = "Something went wrong" — and the exact number often hints at what went wrong.

Successful mkdir:

```
└─(kali㉿kali)-[~/home/file1] └─$ mkdir Udupi
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 0
```

```
└─(kali㉿kali)-[~/home/file1] └─$
```

Unsuccessful mkdir:

```
└─(kali㉿kali)-[~/home/file1] └─$ mkdir test mkdir: cannot create directory 'test':  
File exists
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 1 Successful date:
```

```
└─(kali㉿kali)-[~/home/file1] └─$ date
```

```
Tue Aug 11 07:04:28 AM EDT 2026
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 0
```

Unsuccessful date:

```
└─(kali㉿kali)-[~/home/file1] └─$ date -d date: option requires an argument -- 'd'  
Try 'date --help' for more information.
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 1
```

Successful touch:

```
└─(kali㉿kali)-[~/home/file1] └─$ touch file1.txt
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 0
```

Unsuccessful touch:

```
└─(kali㉿kali)-[~/home/file1] └─$ touch file4 folder/lab1.txt touch: cannot touch  
'folder/lab1.txt': No such file or directory
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 1
```

Successful ls:

```
└─(kali㉿kali)-[~/home/file1] └─$ ls  
file1.txt file4 file4lab1.txt lab lab1.txt Manipal test Udupi
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 0
```

Unsuccessful ls:

```
└─(kali㉿kali)-[~/home/file1] └─$ ls file3 ls: cannot access 'file3': No such file or  
directory
```

```
└─(kali㉿kali)-[~/home/file1] └─$ echo $? 2
```

Successful cd:

```
└─(kali㉿kali)-[~] └─$ cd file
```

```
└─(kali㉿kali)-[~/file] └─$ echo $?  
0
```

Unsuccessful cd:

```
└─(kali㉿kali)-[~/file] └─$ cd file2 cd: no such file or directory: file2
```

```
└─(kali㉿kali)-[~/file] └─$ echo $? 1
```